27 FEB 2026

AADK TASK 2

Designing Interactive Screen with Multiple Buttons & States

MindMatrix

Example App: Lemonade Maker (Multi-Button Version)

1. Feature Idea Chosen

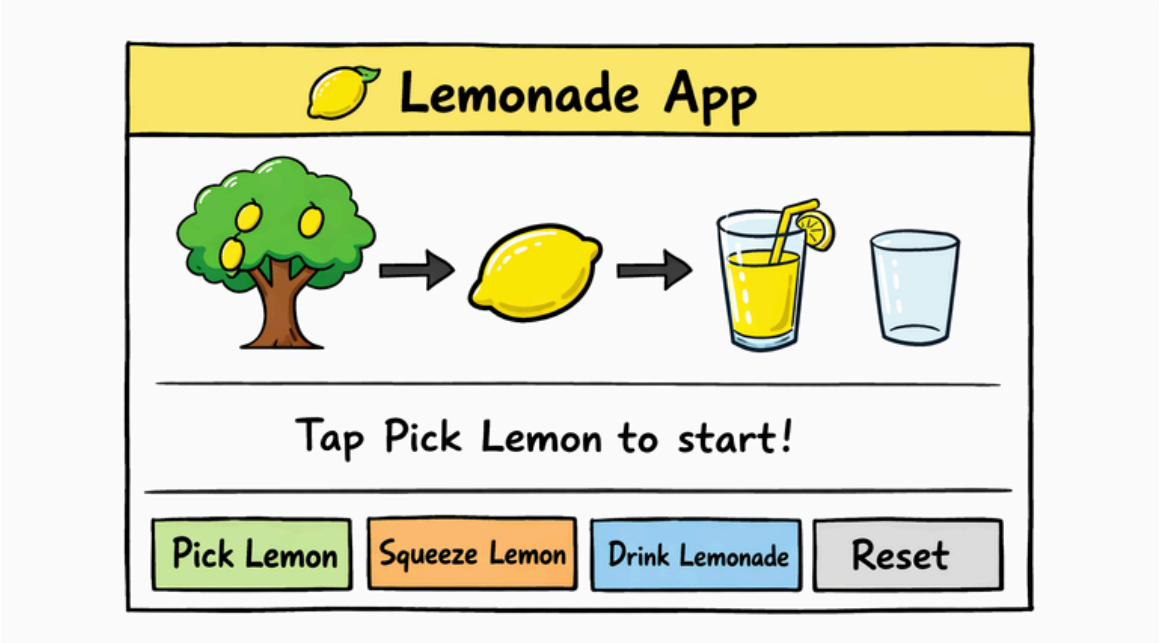
I designed a Lemonade Maker app inspired by the lemonade learning project.

This version includes multiple buttons that change the state of the app:

- 🍋 Pick Lemon
- ✋ Squeeze Lemon
- 🥤 Drink Lemonade
- 🔄 Reset

Each button changes the app's state and updates the UI accordingly.

2. Screen Layout Sketch



UI Elements Included:

- Four distinct buttons (labelled clearly)
- Image area (changes depending on state)
- Status text (shows current instruction or feedback)

3. Button Actions & Associated State Changes

Button A: Pick Lemon

- State Change: state = "LEMON_PICKED"
- A lemon is picked from the tree.
- The image changes to show a lemon.
- Status text updates to: "Now squeeze the lemon!"

Button B: Squeeze Lemon

- State Change: squeezeCount--
- Each press reduces remaining squeezes.
- When squeeze count reaches 0: state = "LEMONADE_READY"
- UI Feedback:

  Text shows remaining squeezes.

  When done: "Lemonade is ready! Drink it."

Button C: Drink Lemonade

- State Change: state = "GLASS_EMPTY"
- Image changes to empty glass.
- Text updates to: "All done! Press Reset to start again."

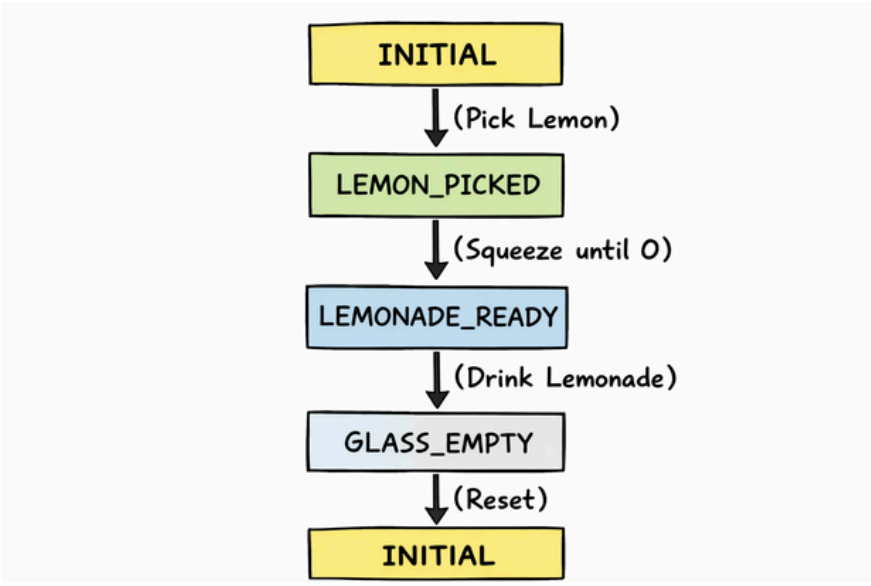
Button D: Reset

- State Change: state = "INITIAL"
- Resets squeeze count.
- UI goes back to original lemon tree screen.
- Text: "Tap Pick Lemon to start!"

4. State Table (Minimum 3 States Included)

Current State	Trigger Button	Resulting UI Change
INITIAL	Pick Lemon	Shows lemon image, instruction to squeeze
LEMON_PICKED	Squeeze Lemon	Decreases squeeze count, updates text
LEMONADE_READY	Drink Lemonade	Shows empty glass image
GLASS_EMPTY	Reset	Returns to lemon tree screen

5. Example State Flow



This shows a clear state transition cycle.

Reflection

If state is not managed properly, the UI can become inconsistent — for example, showing a lemon image while the state says “empty glass.” Poor state handling may also cause stale values (like squeeze count not resetting). To avoid this, I will use proper state variables and reset logic carefully. I will test the app by rotating the device (portrait/landscape) and checking whether the state persists correctly. I will also test rapid button presses to ensure transitions remain valid.